ELI Coding Agent — Test Prompt List

Contents

A categorised prompt suite for exercising the `eli/coding/` engine end-to-end against your **live local model** (the deterministic machinery is already covered by `tests/test_coding_agent.py`; this suite tests real generation quality + the plan→search→verify→repair→memory loop under a real model).

How to run

Option A — direct (recommended, exercises the whole engine + provenance):

```
from eli.coding import solve
r = solve("Implement binary search over a sorted list; return index or -1.", language="python"
print("solved:", r["solved"], "score:", r["score"], "bug_class:", r["bug_class"])
print(r["search"]["explored"], "candidates,", r["search"]["iterations"], "refinements")
print(r["code"])
```

Option B — via the executor action (the wired path):

```
from eli.execution.executor_enhanced import execute
out = execute("CODE_SOLVE", {"description": "<prompt>", "language": "python"})
print(out["solved"], out["score"], out["script_path"])   # saved under artifacts/scripts/
```

Useful env knobs while testing: - `ELI_CODING_BEAM` (default 3) — root candidates (exploration width) - `ELI_CODING_MAX_ITERS` (default 6) — refinement budget - `ELI_CODING_RUN_TIMEOUT` (default 20s) — per-execution wall clock - `ELI_CODING_SANDBOX=0` — disable execution (static-only; for A/B comparison)

For each run, inspect: `solved`, `score`, `search.trace` (per-candidate score/tests/gate), `bug_class`, and the bug-memory growth (`from eli.coding.bug_memory import get_bug_memory; get_bug_memory().stats()`).

---

1. Baseline correctness (should solve cleanly, few/no refinements)

1. "Write `add(a, b)` that returns the sum, with a `main()` demo."
2. "Implement `is_palindrome(s)` ignoring case and non-alphanumerics."

3. “Implement binary search over a sorted list; return the index or -1.”
4. “Write `fizzbuzz(n)` returning a list of strings for 1..n.”
5. “Implement `flatten(nested_list)` for arbitrarily nested lists.”

2. Edge-case heavy (stresses test synthesis + verification gating)

6. “Implement `safe_divide(a, b)` that returns None on division by zero and handles non-numeric input.”
7. “Write `parse_int_list(s)` that turns a comma string into ints, skipping blanks/whitespace, raising ValueError on garbage.”
8. “Implement an LRU cache class with get/put and a capacity bound; evict least-recently-used.”
9. “Write `merge_intervals(intervals)` that merges overlapping [start,end] pairs; handle empty input and single intervals.”

3. Bug-class triggers (stresses semantic classification + repair + bug memory)

(Intentionally under-specified or trap-laden so first attempts often fail and the search must repair — watch `bug_class` and the refinement trace.) 10. “Compute the average of a list of numbers; it must not crash on an empty list.” (→ ZeroDivision / null handling) 11. “Return the last element of a list; handle the empty case.” (→ index out-of-bounds / off-by-one) 12. “Look up a user’s age from a dict by name; handle missing names.” (→ key-missing) 13. “Read a text file and return its line count; handle a missing path.” (→ io_error) 14. “Recursively compute factorial; must handle n=0 and large n without crashing.” (→ recursion / base-case)

4. Analytical / scientific (stresses multi-function decomposition + plots; matplotlib runs headless via Agg)

15. “Estimate π via Monte Carlo with 100k samples; print the estimate and absolute error.”
16. “Fit a line to noisy y = 2x + 1 data with numpy least squares; print slope/intercept and plot the fit.”
17. “Simulate a 1D random walk of 1000 steps over 500 trials; report mean displacement and plot the distribution.”
18. “Compute and plot the first 50 terms of the logistic map for r=3.7; label axes.”

5. Multi-step / structured decomposition (stresses the planner)

19. “Build a tiny CSV report: generate 100 rows of (date, category, amount), aggregate totals per category, and print a sorted summary table.”
20. “Implement a command-line word-frequency counter over a string: tokenize, normalise case, drop stopwords, print the top 10 with counts.”

6. Multiple languages (execution-verified only where the interpreter is installed)

21. “Write a bash script that prints disk usage of the current directory sorted by size.” (language=“bash”)
22. “Write a JavaScript function `dedupe(arr)` returning unique values, with a `console.log` demo.” (language=“javascript”) *(If node/bash aren’t installed, the sandbox tolerates it and verification falls back to static gates — note the difference in `search.trace`.)*

7. Long-running / monitor (verifies timeout is treated as “started fine”, not a crash)

23. “Write a script that polls CPU usage every 2 seconds and prints it, with a clean SIGINT handler.” (expect: runs to timeout → accepted)

8. Repair-loop stress (deliberately hard; expect multiple refinements, possibly unsolved)

24. “Implement Dijkstra’s shortest path on a weighted adjacency dict; return distances from a source. Handle disconnected nodes.”
25. “Implement a thread-safe counter incremented from 10 threads 1000× each; the final value must be exactly 10000.” (→ concurrency/state-corruption class if it races)

9. Subtask-DAG decomposition (multi-component; exercises `plan_graph.solve_dag`)

(These have separable components, so expect `search.mode == "dag"` with a multi-node `order`/`layers`. Set `ELI_CODING_DAG=0` to force single-shot for comparison.) 26. “Build a tokenizer, a stopword filter that uses it, and a top-N word-frequency reporter that uses both.” 27. “Implement a small calculator: a tokenizer, a shunting-yard parser that consumes the tokens, and an evaluator that runs the parsed expression.” 28. “Build a CSV loader, a per-category aggregator that consumes the loaded rows, and a summary printer that consumes the aggregates.”

Reaching it from chat (router triggers)

These phrasings route to `CODE_SOLVE` directly in the chat box (no action call needed): - “solve this coding problem: <…>” - “implement <…> and test it” / “write <…> with unit tests” - “use the coding agent to build <…>” - “give me a verified solution for <…>” Plain “write a python function/script to <…>” routes to the lighter `GENERATE_SCRIPT`.

What to look for (interpreting results)

- **`solved: true` + high `score`** on §1–2 confirms the gate ladder + test synthesis work under your model.
- **`bug_class` populated + `iterations > 0`** on §3 confirms classification + the UCB repair loop fire.
- **`search.explored > beam`** confirms tree expansion (not single-shot).
- **`get_bug_memory().stats()` growing across §3 runs** confirms long-term bug/fix memory is accumulating; re-running a similar §3 prompt should recall a prior fix (visible as a “Known fixes…” line fed into refinement feedback).
- **§7 returning `solved`/clean despite no natural exit** confirms timeout-tolerance.
- **§6 with a missing interpreter** confirms graceful degradation (no false crash).

Known limits to keep in mind while testing

- Solution quality scales with the local model; the engine’s *machinery* is fixed.
- Single-file solving only (no multi-file projects yet).
- Execution verification is Python-deepest; other languages need their interpreter.
- `CODE_SOLVE` is an explicit action — it is **not** auto-routed from free chat yet, so drive it via Option A/B above rather than typing a request in the chat box.